DCC Summary

Lexi Aiello

October 7, 2026

Abstract

• The dependent combinator calculus (DCC) is a dependent type theory based on categorical combinators [**CURIEN1986188**] and affine logic.

- DCC is similar to the base calculus of constructions in that types are terms and that there is an infinite sort hierarchy (but no universe-level polymorphism).
- Propositional equality is inductively generated from $refl$ as in intuitionistic type theory.
- DCC has decidable type-checking, global confluence, and strong normalization.
- DCC is implemented via a deep embedding in Lean 4. We have soundness and completeness of the type-checker, and $Judge : Expr \to Expr \to Type$.
- A subset of Lean 4 can be compiled to DCC.
- β-reduction always terminates in DCC due to affine logic. We eagerly β and η-reduce all types in each constructor, rather than assuming a general conversion axiom. This enables extensional equality reflection, but the proof has a *sorry* for subject-reduction.
- η-reduction is based on observing the behavior of a function on typed "holes" which are generated from the domain of the function, then reducing to the "canonical" combinator producing the hole output (similarly to in NbE). Since we do not assume inductive types and since we do not have recursors on $\mathbb{2}$, we believe we may have computable function extensionality, but this fact is very hard to formalize.

1 Base Types

Our base types are the usual as in MLTT.

Definition 1. *We have an infinite sort hierarchy:*

$$\frac{}{Sort_u : Sort_{u+1}} \tag{1}$$

Definition 2. *We have a bottom type:*

$$\frac{}{\mathbb{0} : Sort_0} \tag{2}$$

Definition 3. *We have a top type with one inhabitant:*

$$\frac{}{\mathbb{1} : Sort_0} \tag{3}$$

$$\frac{}{* : \mathbb{1}} \tag{4}$$

Definition 4. *We have a type $\mathbb{2}$:*

$$\frac{}{\mathbb{2} : Sort_1} \tag{5}$$

It has two inhabitants:

$$\frac{}{\mathbb{0}_{\mathbb{2}} : \mathbb{2}} \tag{6}$$

$$\frac{}{\mathbb{1}_{\mathbb{2}} : \mathbb{2}} \tag{7}$$

2 Syntax

The syntax is given in S-expressions for clarity.

Definition 5. *An expression is either:*

- *A base term inhabiting a base type*
- *A base type*
- *A parenthesized expression beginning with a combinator C and containing its points $p_1 \dots p_n$ separated by whitespace: $(C\ p_1 \dots\ p_n)$.*
- *A juxtaposed list of expressions separated by whitespace, denoting an application.*
- *A parenthesized expression*

Example 1. *$(Id\ A)$ is an expression.*

Proof. See 5. A is a point of Id. □

Example 2. *$((Id\ A)x)$ is an expression.*

Proof. See 5. □

3 Core Combinators

DCC is an extension of the simply-typed categorical combinators [**CURIEN1986188**] to dependent types and affine logic. Our choice of the categorical combinators is due to their close correspondence with cartesian closed categories. We support affine logic by "box" and "bang" terms:

Definition 6. *A bang type $(!A)$ is inhabited by a boxed term $(\&x) : (!A)$:*

$$\textsc{Box}\ \frac{x : A}{(\&x) : (!A)} \tag{8}$$

$$\textsc{Bang}\ \frac{A : Sort_u}{(!A) : Sort_u} \tag{9}$$

Combinators' type arguments are given as points of the combinators, but we assume the existence of Σ-types, so expressivity as if type arguments were given by application is recovered since Σ-pairs can encode telescope contexts.

Definition 7. *The head-reduction rules on combinators are as follows:*

$$\textsc{Id}\ \frac{}{(Id\ A)x \to_{\beta_{wh}} x} \qquad \textsc{Fst}\ \frac{}{(Fst\ B)(,B_2\ x\ y) \to_{\beta_{wh}} x} \qquad \textsc{Snd}\ \frac{}{(Snd\ B)(,B_2\ x\ y) \to_{\beta_{wh}} y}$$

$$\textsc{Curr}\ \frac{}{(\Lambda\ A\ B\ x)yz \to_{\beta_{wh}} (x(,B\ y\ z))} \qquad \textsc{Comp}\ \frac{}{(\circ\ x\ y)z \to_{\beta_{wh}} (x(yz))} \qquad \textsc{Quot}\ \frac{}{(`B\ x)y \to_{\beta_{wh}} x}$$

$$\textsc{Curr'}\ \frac{}{(!\Lambda\ A\ B\ x)(\&\ x)(\&\ y) \to_{\beta_{wh}} (x(\&\ (,B\ y\ z)))} \qquad \textsc{Pair}\ \frac{}{(\langle,\rangle B\ x\ y)(\&\ x) \to_{\beta_{wh}} (,(Bx)\ (Mx)\ (Nx(Mx)))}$$

$$\textsc{Eval}\ \frac{}{(Eval\ A\ B\ x)(,B_2\ y\ z) \to_{\beta_{wh}} xyz}$$

$$\textsc{Unbox}\ \frac{}{(*\ A)(\&\ x) \to_{\beta_{wh}} x} \qquad \textsc{Bang map}\ \frac{}{(!<>\ f)(\&x) \to_{\beta_{wh}} (!(fx))} \qquad \textsc{Box map}\ \frac{}{(\$<>\ f)(\&x) \to_{\beta_{wh}} (\&(fx))}$$

$$\textsc{Eql}\ \frac{}{(Eql\ B)x(,B_2\ y\ z) \to_{\beta_{wh}} (=\ (Bx)\ y\ z)} \qquad \textsc{Sigma}\ \frac{}{(\Sigma\ B\ C)x \to_{\beta_{wh}} (\Sigma(Cx))}$$

DUP

$$\overline{(Dup\ B\ x)(\&y)(\&z) \to_{\beta_{wh}} (xy(,(`(By)\ (By))\ z\ z))}$$

There are also some basic operations on un-boxed Σ-pairs added as optimizations to avoid boxing the term and using the pair rule:

BIMAP

$$\overline{(Map\ A\ B\ f\ g)(,B_2\ x\ y) \to_{\beta_{wh}} (,(`A\ B)\ (fx)\ (gy))}$$

ASSOC

$$\overline{(Assoc\ A\ B\ C)(,B_2\ (,B_3\ x\ y)\ z) \to_{\beta_{wh}} (,(`A\ (\Sigma(`BC)))\ x\ (,(`B\ C)\ y\ z))}$$

SWAP

$$\overline{(Swap\ A\ B)(,Bxy) \to_{\beta_{wh}} (,(`B\ A)\ yx)}$$

DCC uses call-by-value semantics, so a term is reduced by either its head-reduction, reducing the points of the term if it is a combinator, or the usual congruences on functions and arguments.

Definition 8. *The full β-reduction rules are as such:*

$$\frac{f \to_\beta f'}{(fx) \to_\beta (f'x)}\ \text{CONGR FUN.} \qquad \frac{x \to_\beta x'}{(fx) \to_\beta (fx')}\ \text{CONGR APP.} \qquad \frac{(fx) \to_{\beta_{wh}} B}{(fx) \to_\beta B}\ \text{HEAD}$$

$$\frac{points(f,n) \to_\beta p'}{f \to_\beta points.set(f,n,p')}\ \text{CONGR POINT}$$

Example 3. $(,B\ ((Id\ A)x)\ y) \to_\beta (,B\ x\ y)$

Proof. By the congr. point rule since "," is a combinator with point arity 3, and since $(Id\ A)x \to_{\beta_{wh}} x$. □

3.1 Strong Normalization by Syntactic Size Measure

DCC is provably strongly normalizing by a syntactic size measure on expressions, since the rules that duplicate a term expect that term to be boxed (e.g., the *Pair* rule). We simply count the size of a boxed term thrice so that the offending rule decreases the size of the term ($size(\&x) \coloneqq size(x) * 3 + 1$). The size measure is relatively obtuse and arbitrary, but strong normalization is a trivial result of affine logic, so the details have been omitted. We separately show that DCC is locally confluent, and thus inherit global confluence by Newmann's Lemma.

Since strong normalization under β-reduction is always provable by our measure, β-equality is decidable. We have a β-reduction function with soundness, and completeness up to normal forms.

4 η-reduction

DCC is a minimal calculus, and we do not have inductives or recursors for 2. So, a function f is η-reduced by characterizing its behavior on a "hole" matching the shape of its domain. A hole is either an opaque term indexed by a unique level ($hole(n)$), a Σ-pair containing two holes, or a boxed hole. The η-reduct of f is defined to be the canonical combinator yielding the β-normal form of $(fhole(Dom(f)))$ with its domain substituted back to the domain of f.

Example 4. $(\circ(Fst\ B)(Id\ (\Sigma B))) \to_\eta (Fst\ B)$

Proof. This is since $hole(Dom(\circ(Fst\ B)(Id\ (\Sigma B)))) = hole(\Sigma B) = (hole(Dom(B)), hole((Dom(B)hole(Dom(B)))))$ and $(\circ(Fst\ B)(Id\ (\Sigma B))) \to_\beta hole(1)$. So, the η-reduct is defined to be $(Fst\ B)$. □

We have omitted the full details of η-reduction for brevity in this summary.

5 Inference Rules

Since DCC is strongly normalizing, we eagerly reduce $(\eta \circ \beta)$ each type in the consequent of each inference rule, rather than allowing a catch-all conversion rule. This theoretically permits equality reflection, since $h : (=\ B\ x\ y)$ iff $x \equiv y$, but we have a "sorry" in the proof for subject-reduction.

5.1 Definitional Equality

Definition 9. *We define definitional equality as such:*

$$\frac{A \to_\beta^* B \qquad B \text{ is a } \beta\text{-normal form} \qquad B \to_\eta^* C}{A \equiv_\to C} \quad \text{Reduction to Normal Form} \tag{10}$$

$$\frac{A \equiv_\to C \qquad B \equiv_\to C}{A \equiv B} \quad \text{Definitional Equality} \tag{11}$$

5.2 Application and Π

DCC is a pure calculus, so there is no distinction between arrows and Π-types, nor between Σ and products.

Definition 10. *The inference rule for Π is as such:*

$$\frac{B : (env : A) \to Sort_u \qquad C : (env : A) \to (x : B\ env) \to Sort_v \qquad ((env : A) \to Sort_{\max(u,v)}) \equiv_\to \tau}{(\Pi\ B\ C) : \tau} \tag{12}$$

$(env : A) \to (x : B\ env) \to Sort_v$ is written in pure DCC notation as:

$$((\Pi\ (Id(Sort\ o))(`(Sort\ o)(\Pi\ B\ (\Lambda\ A\ B\ (`(\Sigma\ B)\ (Sort\ n))))))A) \tag{13}$$

Definition 11. *The inference rule for application is as such:*

$$\frac{f : ((\Pi\ B\ C)\ env) \qquad x : A \qquad (B\ env) \equiv A \qquad (C\ env\ x) \equiv_\to \tau}{(fx) : \tau} \quad \text{App} \tag{14}$$

5.3 Other Inference Rules

Refer to 1 for the base types. I will keep this summary brief, so I have listed some of the most important inference rules and their expansion as pure DCC syntax here. For the remaining rules, I implicitly reduce the consequent type in each inference rule with $\equiv_\to$ for brevity.

Definition 12. *Arrows are defined via a DSL as Π types that discard their domain with quotation:*

$$(\to\ \ (:\ A\ \tau_A)\ B) \coloneqq ((\Pi\ (Id\ \tau_A)\ (`\tau_A\ (`A\ B)))\ A) \tag{15}$$

Definition 13. *The inference rule for Σ is as such:*

$$\frac{A : Sort_u \qquad B : (x : A) \to Sort_v}{(\Sigma\ B) : Sort_{\max(u,v)}} \tag{16}$$

Definition 14. *Sigma pairs are defined as Σ types that discard their Fst component:*

$$(\times AB) \coloneqq (\Sigma\ (`A\ B)) \tag{17}$$

Definition 15. *The inference rule for Λ is as such:*

$$\frac{A : Sort_u \qquad B : A' \to Sort_v \qquad A \equiv A' \qquad x : (x : (\Sigma B')) \to Cx \qquad B \equiv B'}{(\Lambda\ A\ B\ x) : (x : A) \to (y : Bx) \to ((\Lambda\ A\ B\ C)xy)} \tag{18}$$

In pure DCC syntax, $(x : A) \to (y : Bx) \to (\Lambda\ A\ B\ C)$ is written as:

$$((\Pi(Id\ (Sort\ u))(`(Sort\ u)(\Pi B(\Lambda ABC))))A) \tag{19}$$

Definition 16. *The inference rule for* $!\Lambda$ *is as such:*

$$\frac{A : Sort_u \qquad B : A' \to Sort_v \qquad A \equiv A' \qquad x : (x : (!(\Sigma B'))) \to Cx \qquad B \equiv B'}{(!\Lambda\ A\ B\ x) : (x : (!A)) \to (y : (!<>\ B)x) \to ((!\Lambda\ A\ B\ C)xy)} \tag{20}$$

In pure DCC syntax, $(x : (!A)) \to (y : (!<>\ B)x) \to ((!\Lambda\ A\ B\ C)xy)$ *is written as:*

$$((\Pi(Id(Sort\ u))(`(Sort\ u)(\Pi(!<>\ B)(!\Lambda ABC))))(!A)) \tag{21}$$

5.3.1 Propositional Equality

Propositional equality is inductively defined by reflexivity. Since we eagerly reduce types in every constructor, rather than permitting a catch-all conversion constructor, we believe equality reflection is provable as a computable metatheorem, but there is a "sorry" for subject-reduction. Subject-reduction is technically challenging to formalize due to call-by-value semantics, which requires reducing the points of all combinators.

Definition 17. *The inference rule for* $(refl\ B)$ *is as such:*

$$\frac{A : Sort_u \qquad B : A \to Sort_v}{(refl\ B) : (env : (!A)) \to (x : ((!<>\ B)env)) \to ((Dup\ B\ (Eql\ B))\ env\ x)} \tag{22}$$

$(env : (!A)) \to (x : ((!<>\ B)env)) \to ((Dup\ B\ (Eql\ B))\ env\ x)$ *is written in pure DCC notation as:*

$$((\Pi(Id(Sort\ u))(`(Sort\ u)(\Pi(!<>\ B)((Dup\ B)(Eql\ B)))))(!A)) \tag{23}$$

Due to our η-reduction procedure based on the behavior of a function on its holes, we can prove equalities between some functions with reflexivity (but the general formal metatheorem is quite challenging):

Example 5. *That reflexivity on* $(\&(Id\ (Sort_1)))$ *is a proof that* $(= \ldots (Id\ (Sort_1))(\circ(Id\ (Sort_1))(Id\ (Sort_1))))$*, in raw DCC:*

```
((Id (= ((\Pi (Id (Sort 2))
              (`(Sort 2) (`(Sort 1) (Sort 1))))
          (Sort 1))
         (Id (Sort 1))
         (\circ (Id (Sort 1)) (Id (Sort 1)))))
 ((refl (Id (Sort 2)))
  (& (\rightarrow (: (Sort 1) (Sort 2))
                  (Sort 1)))
  (& (Id (Sort 1))))) : (= ((\Pi (Id (Sort 2))
                                 (`(Sort 2)
                                   (`(Sort 1) (Sort 1))))
                            (Sort 1))
                           (Id (Sort 1))
                           (Id (Sort 1)))
```

Proof. This example type-checks via the DCC type-checker. We are happy to supply the GitHub repository, although it is private currently. □

6 Compiling Lean 4 to DCC

It is relatively straightforward to compile the subset of Lean 4 that corresponds to the base CoC to DCC (i.e., minus inductive types, quotients, and some syntactic sugar), due to the close correspondence between categorical combinators and cartesian closed categories. There is significantly less bookkeeping required to track contexts than in other calculi (e.g., dependent SK), since they are easily expressed as telescopes via Σ. The main bookkeeping required is reference counting for boxes. This is done with a preprocessing step annotating types in the Lean 4 term with bangs and boxes, depending on their usage count.

At a high level, application MN under a binder corresponds to:

$$(\circ(Eval\ \ldots\ \ldots\ [\![M]\!])(\langle,\rangle\ldots\ (Id\ldots)(\Lambda\ldots\ldots(\circ[\![N]\!](Fst\ldots))))) \tag{24}$$

Debruijn indexing under a binder corresponds to Σ-projection composition. Contexts correspond to the index B in $(Fst\ B)$ and $(Snd\ B)$, and they are always initialized at the top-level of the term with the firstmost element being the unit $*$ to represent an empty context. Lambda binders correspond to the Curry combinator Λ. I have omitted the details for brevity, but we have been able to compile the point-free dependent K combinator using this approach.

7 In Summary

To summarize, DCC in its present form is a minimal calculus with decidable type-checking, strong normalization, and global confluence extending the simply-typed categorical combinators to dependent types and affine logic. It aims to be similar to the base Calculus of Constructions. As of present, we have the following formalized in Lean 4:

- A β-reducer with completeness up to normal forms
- An η-reduction algorithm which observes the behavior of functions based on typed holes to canonicalize functions
- A judgment relation living in $Type$, which eagerly reduces types of all combinators in the respective rules, rather than allowing permissible catch-all conversion
- A proof of global confluence
- A proof of strong normalization using a syntactic size measure
- A type-checker with completeness and soundness with respect to judgments
- An equality reflection metatheorem with a "sorry" for only subject-reduction
- Examples of reflexivity type-checking some basic function extensionalities with the DCC type-checker
- A procedure for compiling the subset of Lean 4 that corresponds to the base CoC to DCC with some examples
  - We can compile the dependent K combinator written as Lean 4 code to DCC

8 Limitations